Application development for Android

ContactsApp

Architecture, Domain Models, MVVM & Type-Safe Navigation

Forms & JSON Persistence

Media, Camera & Coil 3

GPS, Maps & Adaptive UI

Contacts App Roadmap

#1 - Domain & Type-Safe Navigation

- Model & Collator: Accent-aware alphabetical sorting
- MVVM & StateFlow: Reactive stream & stateless screens
- @Serializable Routes: Navigation 2.8

#2 - Forms & JSON Persistence

- UiState & Regex: Real-time email and phone validation
- Material 3 DatePicker
- ContactsRepository: Internal storage JSON & IO dispatch

#3 - Media, Camera & Coil 3

- Scoped Storage & Picker
- FileProvider & Camera: TakePicture contract & sandbox
- AsyncImage (Coil 3): Caching & circular avatar badges

#4 - GPS, Maps & Adaptive UI

- FusedLocationProvider: Runtime permissions & fix stamps
- Maps Compose & OSM: Reactive pins & auto-fitting bounds
- Adaptive Layouts: Split responsive landscape design

#1 - Learning Objectives

- Set up a modern Android project without XML layout files.
- Create pure Kotlin business domain entities isolated from UI frameworks.
- Implement custom locale-aware alphabetical sorting with java.text.Collator.
- MVVM architecture using `ViewModel` and `StateFlow`.
- Configure Jetpack Compose Navigation 2.8+ using serializable class routes.
- Build a stateless virtualized list using LazyColumn with stable keys.

Build Configuration & Version Catalogs

- Modern dependency management via Gradle Version Catalogs
- Applying the Kotlin Serialization compiler plugin is mandatory for Type-Safe Navigation

libs.versions.toml

```
[plugins]
. . .
kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization", version.ref = "kotlin" }
```

build.gradle.kts (Project)

```
plugins {
. . .
alias(libs.plugins.kotlin.serialization) apply false
```

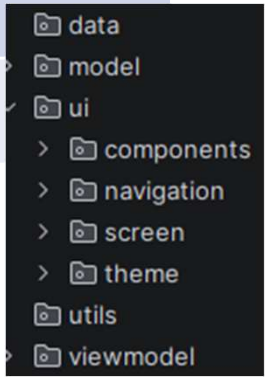
build.gradle.kts (Module)

```
plugins {
. . .
alias(libs.plugins.kotlin.serialization)
```

Project Organization

- Create an *Android* project choosing "*Empty Activity”*
- Structure packages by layer responsibility:

Package	Responsability	Example
`model`	Pure Kotlin data classes (no Android imports)	`Contact, ContactsList, MeetingPoint`
`viewmodel`	UI state and presentation logic	`ContactsViewModel, EditContactUiState`
`data`	Provides access to data and repositories.	`ContactsRepository`
`utils`	Reusable helper functions and classes	`FileUtils, LocationTracker`
`ui.navigation`	Type-Safe route definitions	`ContactListRoute, ContactDetailsRoute, EditContactRoute`
`ui.components`	Reusable stateless UI building blocks	`ContactCard, ContactAvatar, ContactMeetingMap`
`ui.screens`	Screen composables and page containers	`ListScreen, ShowScreen, EditScreen`
`ui.theme`	Material 3 design system and typography.	`Color.kt, Theme.kt, Type.kt`



Domain Modeling
Contact.kt

- A contact will be defined by
  - Name
  - E-mail
  - Phone
  - Birthday (can be `null`)
  - Picture (can be `null`)

model

Contact.kt

```
package pt.isec.amov.contacts.model

import kotlinx.serialization.Serializable
import java.util.UUID

@Serializable
data class Contact(
    val id: String = UUID.randomUUID().toString(),
    val name: String,
    val email: String,
    val phone: String,
    val birthdayEpochMillis: Long? = null,
    val picturePath: String? = null
)
```

The Accent Sorting Challenge in Portuguese

- Standard `sortedBy { it.name }` relies on raw ASCII/Unicode byte values.
- Problem: Álvaro or Ângela get sorted after Nuno.

```
val names = listOf("Ñoño", "Nuno", "Änne", "Ana", "Álvaro", "andré", "Ângela")

// Without Collator: natural Unicode ordering
val withoutCollator = names.sorted()
```

→

```
Ana
Nuno
andré
Álvaro
Ângela
Änne
Ñoño
```

- Solution: Use `java.text.Collator` with **primary** strength to disregard diacritics and case during sorting.

```
// With Collator: language-aware alphabetical ordering
val primary = Collator.getInstance(Locale.getDefault())
primary.strength = Collator.PRIMARY
```

→

```
Álvaro
Ana
andré
Ângela
Änne
Nuno
Ñoño
```

Domain Collection
ContactsList.kt

- Create a class to represent the list of contacts.
- Implement Core functions:
  - Locale-aware sorting `getAllSorted(): List<Contact>`
  - Find and return the first Contact matching the specified id `findById(id: String): Contact?`
  - Append the provided Contact instance to the internal list `add(contact: Contact)`
  - Immutable snapshot copy of contacts for safe data persistence `rawList(): List<Contact>`

model

ContactsList.kt

```
package pt.isec.amov.contacts.model

import java.text.Collator
import java.util.Locale

class ContactsList(initialContacts: List<Contact> = emptyList()) {
    private val _contacts = initialContacts.toMutableList()

    fun getAllSorted(): List<Contact> {
        val collator = Collator.getInstance(Locale.getDefault()).apply {
            strength = Collator.PRIMARY
        }
        return _contacts.sortedWith { c1, c2 -> collator.compare(c1.name, c2.name) }
    }

    fun findById(id: String): Contact? = _contacts.find { it.id == id }
    fun add(contact: Contact) { _contacts.add(contact) }
    fun rawList(): List<Contact> = _contacts.toList()
}
```

ViewModel
ContactsViewModel.kt

- Create a `ViewModel` class:
  - Encapsulate a private instance of the domain class `ContactsList` for domain initialization with seed data (3 contacts)
  - Declare a private mutable property `_contactsList` and expose it as public read-only `contactList` initialized with the sorted list
  - Implement the public member function `getContactById(id: String): Contact?`, delegating to `contactsListModel.findById(id)` to retrieve a specific contact by its unique identifier.

viewmodel

ContactsViewModel.kt

```
class ContactsViewModel : ViewModel() {
    private val contactsListModel = ContactsList(
        listOf(
            Contact(name = "Álvaro Lopes", email = "alopes@isec.pt", phone = "912345678"),
            Contact(name = "Beatriz Silva", email = "bsilva@isec.pt", phone = "987654321"),
            Contact(name = "Ana Martins", email = "amartins@isec.pt", phone = "924681357")
        )
    )

    private val _contactsList = MutableStateFlow(contactsListModel.getAllSorted())
    val contactsList: StateFlow<List<Contact>> = _contactsList.asStateFlow()

    fun getContactById(id: String): Contact? = contactsListModel.findById(id)
}
```

UI Interface

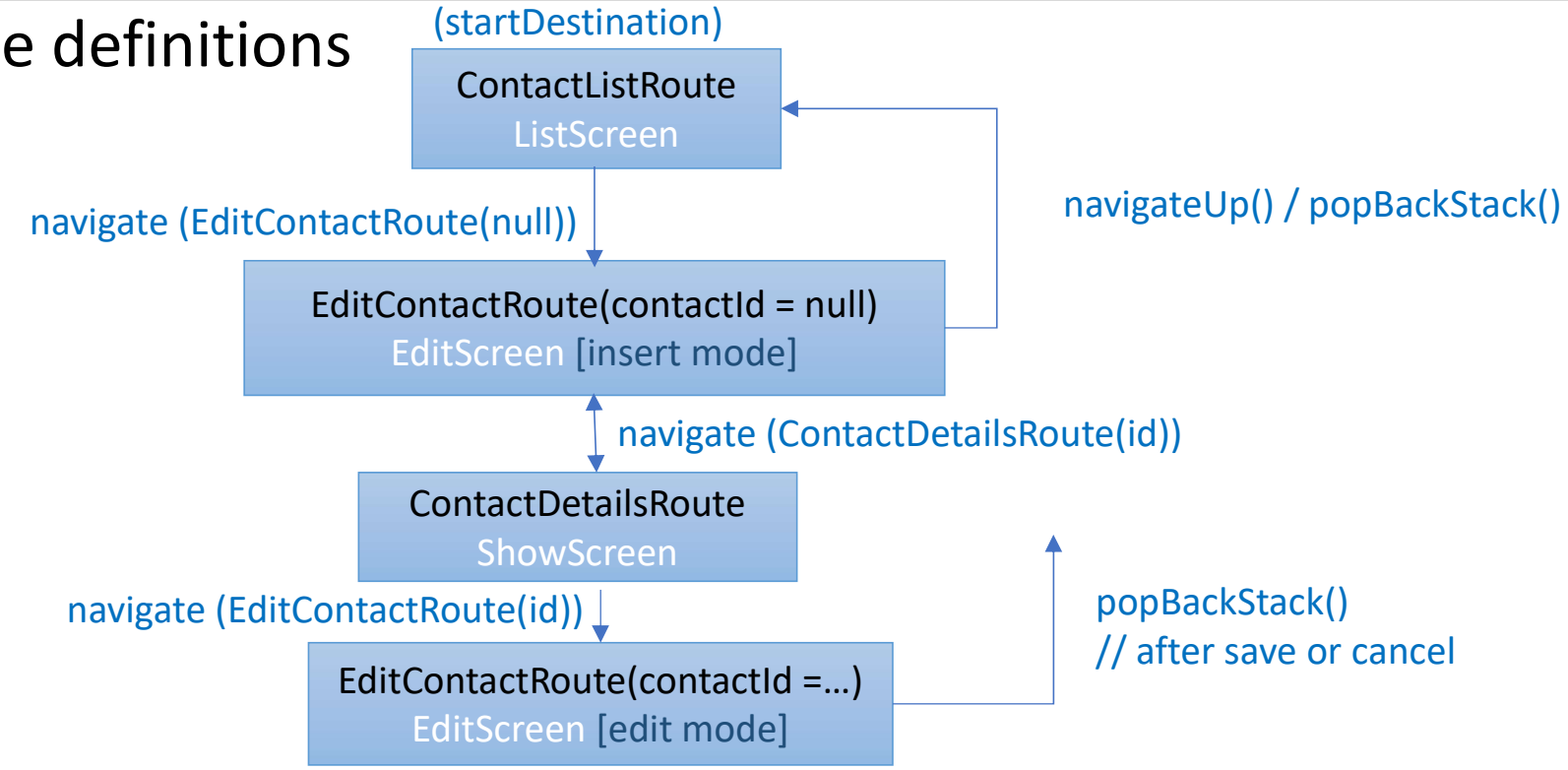
- The application should display a **main screen** with a list of contacts
  - This list should present each contact as a `Card`
  - By default, only the contact's name will be shown
    - Include an `IconButton` in the `TopBar` to switch between this view and an expanded view to show more of the contact's information
- When a `Card` is clicked, a **new screen** should display all the information related to the selected contact
  - On this screen, an *Edit* `IconButton` in the `TopBar` should allow the user to edit the contact's information
- In the main screen, an *Add* `IconButton` should enable the user to **create a new contact**, using a new screen for this operation
  - Use the **same screen for creating a new contact or editing** an existing one
  - On this screen, a *Done* `IconButton` should confirm the changes, update the global contact list, and return the user to the main screen
- All screens should **display a back navigation icon** (back arrow) in the `TopBar`, except the main screen

Type-Safe Navigation vs. Legacy String Routes

- Legacy Compose Navigation used error-prone string concatenation: `"details/{contactId}"`.
  - Typos and mismatched argument types could cause runtime crashes
- **Navigation 2.8+** introduces compile-time safe navigation with `@Serializable` classes and objects.
  - Routes are defined as strongly typed Kotlin structures, enabling compile-time safety and eliminating manual string concatenation.
  - Dedicated File & Top-Level Declaration
    - Keep all route definitions centralized in a single dedicated file (e.g. `Destinations.kt` or `Routes.kt`) located inside the ui/navigation package
    - Declare each route directly at the top level of the file
  - Mandatory `@Serializable` Annotation
    - Annotate **every single route**
    - Allow safely encode route parameters into Android's navigation `backstack` and `SavedStateHandle`
  - Use `data object` for Parameterless Routes (e.g., Main/List Screen)
  - Use `data class` for Parameterized Routes
    - It provides automatic `toString()`, `equals()`, and `hashCode()` implementations, which Navigation Compose uses to identify destinations cleanly.

Declaring Routes
Destinations.kt

- Top-level route definitions




```
package pt.isec.amov.contacts.ui.navigation

import kotlinx.serialization.Serializable

@Serializable
data object ContactListRoute

@Serializable
data class ContactDetailsRoute(val contactId: String)

@Serializable
data class EditContactRoute(val contactId: String? = null)
```

ui.navigation

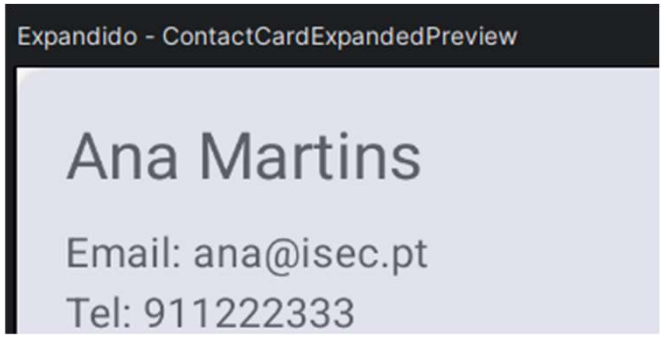
Destinations.kt

Stateful vs. Stateless Composables

- **Stateless Composables:**
  - Receive plain immutable data
  - Emit events upwards through lambdas.
  - Easy to preview and unit test.
  - Holds no internal business logic or persistent state.
  - This decouples the view from any data source, database, or ViewModel
- **Stateful Composables:**
  - Connected to navigation or ViewModels
  - host state and forward events downwards.

Implement Expandable `ContactCard` component

- Create a stateless `@Composable fun ContactCard` that takes
  - a `contact: Contact`
  - an `isExpanded: Boolean flag`
  - an `onClick: () -> Unit callback`
  - a default `modifier: Modifier`
- It must render a clickable, full-width `Card` with `2.dp` elevation and `16.dp` padding:
  - Display the contact's name using `MaterialTheme.typography.titleLarge`.
  - Wrap the email and phone fields inside an `AnimatedVisibility(visible = isExpanded)` block with `8.dp` top padding, rendering each with `MaterialTheme.typography.bodyMedium`





Building a Stateless Item Card
ContactCard.kt

- Supporting card expansion with smooth animated transitions

ui.components

ContactCard.kt

```
@Composable
fun ContactCard(
    contact: Contact,
    isExpanded: Boolean,
    onClick: () -> Unit,
    modifier: Modifier = Modifier
) {
    Card(
        modifier = modifier
            .fillMaxWidth()
            .clickable(onClick = onClick),
        elevation = CardDefaults.cardElevation(defaultElevation = 2.dp)
    ) {
        Column(modifier = Modifier.padding(16.dp)) {
            Text(text = contact.name, style = MaterialTheme.typography.titleLarge)
            AnimatedVisibility(visible = isExpanded) {
                Column(modifier = Modifier.padding(top = 8.dp)) {
                    Text(
                        text = "Email: ${contact.email}",
                        style = MaterialTheme.typography.bodyMedium
                    )
                    Text(
                        text = "Tel: ${contact.phone}",
                        style = MaterialTheme.typography.bodyMedium
                    )
                }
            }
        }
    }
}
```

Implement `ListScreen`

- Create a stateless `@Composable fun ListScreen` that receives:
  - `contacts: List<Contact>`
  - `isExpanded: Boolean`
  - `onContactClick: (String) -> Unit`
  - `modifier: Modifier = Modifier`
- Requirements:
  - **Empty State:** If `contacts` is empty, display a centered `Box` filling the maximum size (`Alignment.Center`) with the message "`No contacts registered.`".
  - **Virtualized List:** If the collection contains items, render a `LazyColumn` filling the entire available size, configured with:
    - `contentPadding = PaddingValues(16.dp)`
    - `verticalArrangement = Arrangement.spacedBy(8.dp)`
- Stable Identity & Items:
  - Iterate over the collection using `items` and set a stable key with `key = { it.id }`.
  - For each contact, instantiate a `ContactCard`, forwarding `contact`, `isExpanded`, and invoking `onContactClick(contact.id)` upon click.

Screen Implementation
ListScreen.kt

- Stateless list container

ui.screens

ListScreen.kt

```
@Composable
fun ListScreen(
    contacts: List<Contact>,
    isExpanded: Boolean,
    onContactClick: (String) -> Unit,
    modifier: Modifier = Modifier
) {
    if (contacts.isEmpty()) {
        Box(modifier = modifier.fillMaxSize(), contentAlignment = Alignment.Center) {
            Text("No contacts registered.")
        }
    } else {
        LazyColumn(
            modifier = modifier.fillMaxSize(),
            contentPadding = PaddingValues(16.dp),
            verticalArrangement = Arrangement.spacedBy(8.dp)
        ) {
            items(items = contacts, key = { it.id }) { contact ->
                ContactCard(
                    contact = contact,
                    isExpanded = isExpanded,
                    onClick = { onContactClick(contact.id) })
            }
        }
    }
}
```

Implement `ShowScreen` for Contact Detais

- Create a stateless `@Composable fun ShowScreen` with the following parameters:
  - `contact: Contact?`
  - `onEditClick: (String) -> Unit`
  - `modifier: Modifier = Modifier`
- Requirements:
  - **Fallback for Missing Contact:** If `contact == null`, display a full-screen centered `Box (Alignment.Center)` with the message "`Contact not found.`" and exit immediately using an early return.
  - **Main Structure:** If the contact exists, render a `Scaffold`:
    - Include a `FloatingActionButton` containing an `Icons.Default.Edit` icon that invokes `onEditClick(contact.id)` when clicked.
- **Content Layout:** Inside the scaffold content, display a `Column` that fills the maximum size, applying both the scaffold's inner `padding` and an additional `16.dp` padding:
  - Display the name with "`Name: ${contact.name}`" using `MaterialTheme.typography.titleLarge`
  - Display the email with "`Email: ${contact.email}`" using `MaterialTheme.typography.bodyLarge`
  - Display the phone number with "`Phone: ${contact.phone}`" using `MaterialTheme.typography.bodyLarge`

Implement Dual-Purpose `EditScreen` Placeholder

- Create a stateless `@Composable fun EditScreen` that accepts:
  - `contactId: String?`
  - `onBack: () -> Unit`
  - `modifier: Modifier = Modifier`
- Requirements:
  - Root Container: Place a `Box` that fills the maximum size (`fillMaxSize()`) with its content aligned strictly at the center (`contentAlignment = Alignment.Center`).
  - Form Layout: Inside the `Box`, render a `Column` with horizontal alignment set to `Alignment.CenterHorizontally`:
    - **Dynamic Title:** Display a `Text` composable showing `"Create Contact"` if `contactId == null`, or `"Edit Contact: $contactId"` if a non-null identifier is received.
    - **Spacing:** Insert a `Spacer` with a vertical height of `16.dp`.
    - **Navigation Action:** Add a `Button` labeled "Back" that invokes the `onBack` callback when clicked

Implementation of Contextual `TopAppBar`

- **Function Parameters:**
  - `navController: NavHostController`: Manages backstack tracking and back-navigation.
  - `isExpanded: Boolean`: Controls card expansion (expanded/compact)
  - `onToggleExpand: () -> Unit`: Callback triggered when toggling the card display mode.
  - `onAddContact: () -> Unit`: Callback triggered when requesting the addition of a new contact.
- **Reactive Destination Tracking (Type-Safe Query):**
  - Observe the current backstack entry reactively using `navController.currentBackStackEntryAsState()`.
  - Determine the boolean flag `isList` by verifying via type-safe checking whether the current destination corresponds to `ContactListRoute (navBackStackEntry?.destination?.hasRoute<ContactListRoute>() == true)`.
- **TopAppBar Structure and Actions:**
  - **Title:** Display a `Text` composable with `"Contacts"`.
  - **Navigation Icon (`navigationIcon`):** If the current screen is **not** the main list (`!isList`), display na `IconButton` with the icon `Icons.AutoMirrored.Filled.ArrowBack` (description "`Back`"), which executes `navController.navigateUp()` on click.
  - **Action Buttons (`actions`):** Only when situated on the main list screen (`isList`), render two action buttons:
    - An `IconButton` connected to `onToggleExpand`, toggling between `Icons.Default.ViewAgenda` (if `isExpanded` is true) and `Icons.AutoMirrored.Filled.List` (if false), with the content description "`Toggle`".
    - An `IconButton` connected to `onAddContact`, featuring the `Icons.Default.Add` icon with the description "`Add`"

Configure `MainActivity` with `ViewModel` and Navigation Scaffold

- State Initialization:
  - Instantiate the shared ViewModel using the lifecycle factory:
    `val viewModel: ContactsViewModel = viewModel()`
  - Initialize the navigation controller:
    `val navController = rememberNavController()`
  - Declare a surviving UI state for card expansion:
    `var isExpanded by rememberSaveable { mutableStateOf(false) }`
- Scaffold Layout Structure:
  - Render a `Scaffold` composable with its topBar slot hosting `MainTopAppBar`, providing:
    - `navController = navController`
    - `isExpanded = isExpanded`
    - `onToggleExpand = { isExpanded = !isExpanded }`
    - `onAddContact = { navController.navigate(EditContactRoute(null))}`
  - Pass the Scaffold's `innerPadding` parameter to the trailing lambda placeholder reserved for the root `NavHost` routing definition

Configure the Navigation Host (`NavHost`)

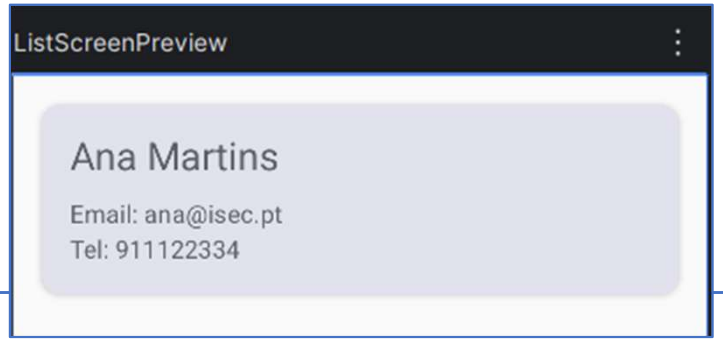
Set up a type-safe `NavHost` using `navController`, `startDestination = ContactListRoute`, and `Modifier.padding(innerPadding)`:

- `composable<ContactListRoute>`:
  - Collect the contacts list from the ViewModel with `viewModel.contactsList.collectAsStateWithLifecycle()`
  - Render `ListScreen` passing `contacts, isExpanded`, and navigate to `ContactDetailsRoute(id)` on item click.
- `composable<ContactDetailsRoute>`:
  - Extract the route arguments using `backStack.toRoute<ContactDetailsRoute>()`
  - Fetch the contact via `viewModel.getContactById(route.contactId)`
  - Render S`howScreen` passing the fetched contact and navigate to `EditContactRoute(id)` when clicking edit.
- `composable<EditContactRoute>`:
  - Extract the route arguments using `backStack.toRoute<EditContactRoute>()`
  - Render `EditScreen` passing `route.contactId` and calling `navController.popBackStack()` on the back action.

Tooling: Android Studio @Preview

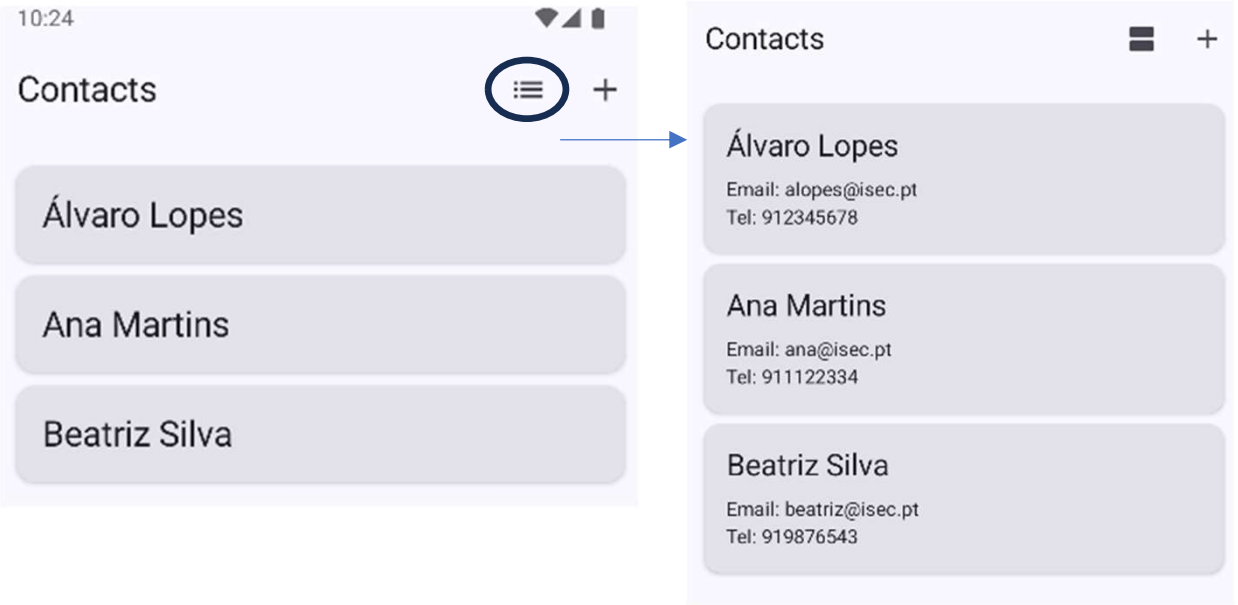
- Testing composables without running an emulator

```
@Preview(showBackground = true)
@Composable
private fun ListScreenPreview() {
    contactsTheme {
        ListScreen(
            contacts = listOf(
                Contact(
                    name = "Ana Martins",
                    email = "amartins@isec.pt",
                    phone = "911122334"
                )
            ),
            isExpanded = true,
            onContactClick = {}
        )
    }
}
```

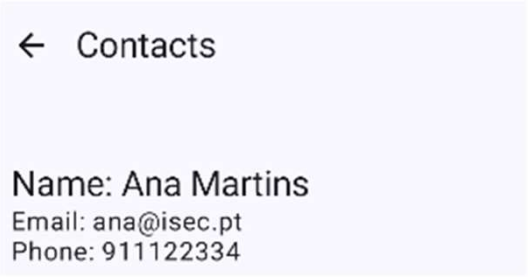


Live Demo & Verification

- Run the app on the emulator or physical device.
- Verify alphabetical sorting: *Álvaro* precedes *Ana*, which precedes *Beatriz*.
- Tap the AppBar toggle button to verify item expansion.



- Tap a contact to navigate to details, then tap the Back button.



- Rotate the device: state remains intact because it lives inside ContactsViewModel
- Explore the Add / Edit functionality (to be completed next week)

AMOV Project

Domain Modeling

- Define clean domain data classes for municipal occurrences (Incident, Category)

```
@Serializable
data class Incident(
    val id: String = UUID.randomUUID().toString(),
    val title: String,
    val description: String,
    val category: String,
    val timestamp: Long = System.currentTimeMillis()
)
```

Navigation Structure

- Declare Type-Safe routes for the municipal dashboard

```
@Serializable data object DashboardRoute
@Serializable data object ReportIncidentRoute
@Serializable data class IncidentDetailRoute(val incidentId: String)
```

Incident Feed

- Build a virtualized incident list using LazyColumn.
- Use card expansion to display incident status (e.g., *Submitted*, *In Review*, *Resolved*).